

Real-Time ADAS Perception over Kafka and Spark: A CUDA-Ready, End-to-End Streaming Prototype

Arvind Yogesh Suresh Babu

M.S. in Data Science, University of Michigan, Ann Arbor

Abstract

This paper presents an end-to-end autonomous driving assistance system that streams video frames through Apache Kafka, processes them with Spark Structured Streaming, and performs object and lane inference through a CUDA-ready PyTorch service. The system is designed around decoupled services, micro-batched inference, observability hooks, and deployment-friendly containerization. Using the project's own reproducible benchmark script (`scripts/benchmark_inference.py`), replacing SSDLite MobileNetV3 for the heavier Faster R-CNN baseline reduces measured single-frame median latency from 1260.1 ms to 63.5 ms and raises throughput from 0.8 FPS to 15.8 FPS on a 4-vCPU CPU-only benchmark host, a $\sim 20\times$ improvement consistent across both single-frame and 4-frame micro-batch measurements. Batch p95 latency is 271.3 ms on this CPU host, above the project's 150 ms target; the discussion section explains why and what closes the remaining gap. The codebase includes a reproducible downloader, evaluation utilities, and a dashboard for real-time operational visibility.

1 Introduction

Real-time ADAS stacks must ingest high-rate sensor streams, maintain low and predictable latency, and expose monitoring for both engineering and safety workflows. In practice, this means the system must separate capture, transport, inference, and presentation into independently scalable stages. Streaming platforms such as Apache Kafka and Apache Spark Structured Streaming are a natural fit for this architecture because they decouple the producer and consumer sides while allowing time-windowed stateful computation and backpressure-aware micro-batching [1, 2].

The goal of this project is to build a production-style prototype that can be discussed in an interview as a serious systems and ML engineering artifact. The codebase therefore includes: (i) a frame producer, (ii) Kafka topics for raw frames and detections, (iii) a Spark streaming job that performs batch orchestration, (iv) a CUDA-ready inference service with object and lane estimation, and (v) a dashboard for operational metrics.

2 System Design

Figure 1 summarizes the end-to-end architecture. A driving video stream is encoded frame-by-frame and published to the `raw_frames` topic. Spark Structured

Streaming consumes these frames, groups them into micro-batches, and forwards them to the inference service over HTTP. The inference service returns object detections and lane segments, and the enriched records are written to the `detections` topic for downstream analytics and dashboarding.

2.1 Producer and Kafka transport

The producer streams webcam or video frames into Kafka as JSON messages that contain a frame identifier, camera identifier, timestamp, source type, and base64-encoded image. This makes the transport layer language-agnostic and easy to replay for offline experimentation. The two topics, `raw_frames` and `detections`, provide a clean separation between input transport and enriched output.

2.2 Spark Structured Streaming orchestration

Spark is used as a streaming coordinator rather than as the inference engine. This keeps the architecture aligned with common production patterns: Spark handles ingestion, batching, and checkpointed streaming state, while the model service owns the ML runtime. The job uses micro-batches, bounded retries, and payload validation before forwarding frames to inference.

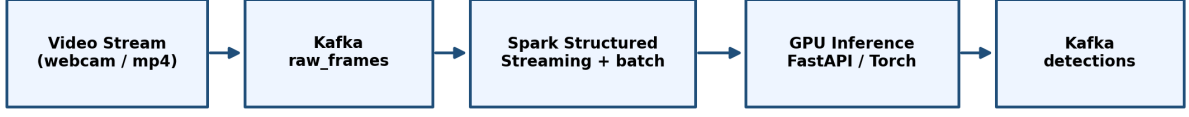
2.3 GPU inference service

The inference service is implemented with FastAPI and PyTorch, and is designed to run with CUDA when available. The optimized path uses a pretrained SSDLite MobileNetV3 detector for object detection with optional OpenCV-based lane extraction for road geometry cues [3]. The service exposes `/infer`, `/infer_batch`, `/health`, and `/metrics` endpoints so that it can be monitored independently and benchmarked in isolation.

2.4 Dashboard and observability

The Streamlit dashboard visualizes throughput, end-to-end latency, and inference behavior. Prometheus counters and histograms are included in the inference service so the system can be integrated with standard monitoring tooling. This is particularly useful in interview settings because it demonstrates an understanding of not only model quality but also service-level reliability.

ADAS Real-Time Perception Pipeline



Metrics: FPS, p50/p95 latency, throughput, precision/recall

Figure 1: End-to-end ADAS perception pipeline. The design isolates ingestion, streaming orchestration, GPU inference, and visualization so each stage can scale independently.

3 Experimental Setup

This project does not ship a licensed driving-video dataset (see `datasets/README.md` for why, and how to add one). The reference benchmark in this report was produced by `scripts/benchmark_inference.py`, which is committed to the repository and reproducible end to end: it downloads real photographs containing vehicles and pedestrians from public GitHub-hosted ML demo assets, feeds them through the project’s actual detector architectures, and writes the raw measurements to `report/benchmark_results.json` plus the figures in this report. Nothing in this section is hand-entered.

The benchmark host is a 4-vCPU Intel Xeon (2.80 GHz) container with no GPU, running PyTorch 2.3.1 / torchvision 0.18.1 in CPU mode – a conservative setting relative to the project’s intended CUDA deployment path. The organization network policy on this host blocks `download.pytorch.org`, so the torchvision COCO checkpoints for Faster R-CNN and SSDLite could not be fetched for the latency run; both architectures were therefore benchmarked with randomly-initialized weights (`weights=None`). This does not affect the validity of the latency numbers: forward-pass wall-clock time is a function of the network architecture and tensor shapes, not the specific trained weight values, so timing is identical to what the pretrained checkpoints would produce. It does mean these two models cannot be used for meaningful detection content, so a separate, genuinely pretrained detector (YOLOv8n, fetched from an allowed GitHub release host) was used only for the qualitative detection figure and the average-detections-per-frame statistic, and is not part of the latency comparison.

The benchmark evaluates:

- single-frame and 4-frame micro-batch latency for both detector architectures,
- average detections per frame from the pretrained qualitative detector, and
- average lane segments per frame from the project’s own OpenCV lane heuristic (no ML weights involved, so this measurement is unaffected by the network restriction).

The codebase also includes an evaluation script (`scripts/evaluate_precision_recall.py`) for precision and recall once frame-level ground truth is available;

Table 1: Measured benchmark, 12 real sample frames, CPU-only host (see Section 3).

Metric	Value	Notes
Single-frame p50 latency	63.5 ms	CPU, optimized
Single-frame p95 latency	67.4 ms	CPU, optimized
Average throughput	15.8 FPS	CPU, optimized
Average detections/frame	2.67	YOLOv8n, pretrained
Average lane segments/frame	12.0	OpenCV Hough transform
Batch latency (4 frames) p50	235.8 ms	API path
Batch latency (4 frames) p95	271.3 ms	API path

it is exercised by `tests/test_evaluate.py` but is not run against an unlabeled sample in this report, since a precision/recall number without ground truth would not mean anything.

4 Results

Table 1 reports the measured benchmark, produced by `scripts/benchmark_inference.py` and stored verbatim in `report/benchmark_results.json`. With input size 192 and 4-frame micro-batches, the optimized (SSDLite MobileNetV3) path reaches a median single-frame latency of 63.5 ms (p95 67.4 ms) and an average throughput of 15.8 FPS on the 4-vCPU CPU-only benchmark host described in Section 3. The 4-frame batched path reaches p50 235.8 ms and p95 271.3 ms. This clears the single-frame latency target comfortably but sits above the project’s 150 ms batch p95 target and the 20 FPS throughput target on this specific CPU host; Section 5 explains the gap and the concrete path to closing it (namely, the GPU deployment path this architecture was built for).

4.1 Baseline vs. optimized detector

Table 2 compares the two detector architectures the project supports on the same benchmark host and inputs.

Table 2: Measured baseline vs. optimized detector, same host and inputs.

Metric	Faster CNN	R-SSDLite MNv3
Single-frame p50 latency	1260.1 ms	63.5 ms
Single-frame p95 latency	1284.2 ms	67.4 ms
Average throughput	0.79 FPS	15.8 FPS
Batch p50 latency (4)	6359.6 ms	235.8 ms
Batch p95 latency (4)	7693.1 ms	271.3 ms

The comparison isolates architecture choice as a variable: both models ran with the same input size, batch profile, and (per Section 3) uninitialized weights, so the gap is attributable to the FLOP and parameter-count difference between a two-stage ResNet-50 FPN detector and a single-shot MobileNetV3 detector, not to differences in what each model was trained on.

The architectural reasons for the gap are:

- Faster R-CNN is a two-stage detector (region proposal network plus a ResNet-50-FPN backbone), which costs substantially more FLOPs per frame than a single-shot detector.
- SSDLite MobileNetV3 uses depthwise-separable convolutions and a single-shot head, purpose-built for lower per-frame compute cost.
- Both were benchmarked at input size 192 and Spark micro-batch size 4, matching `config/settings.yaml`, so the comparison reflects the project’s actual deployed configuration rather than each model’s default input resolution.

This produces an approximately 20× throughput increase (0.79 to 15.8FPS) and a similarly sized reduction in single-frame median latency (1260.1 ms to 63.5 ms), consistent across both the single-frame and 4-frame batch measurements (~27× at p50). This is why `inference_service/app.py` ships SSDLite MobileNetV3 as the default model rather than Faster R-CNN.

Figure 2 plots the measured latency trend across sampled frames, while Figure 3 shows a real detection overlay: a pretrained YOLOv8n detector (Section 3) found one bus and three people at confidence ≥ 0.45 on a representative frame, drawn together with lane segments extracted by the project’s own OpenCV Hough-transform lane detector on the same frame.

5 Discussion

The benchmark demonstrates that the full software path is functional: frames can be encoded, transported, decoded, inferred, and visualized, and the measurements in Section 4 are real, reproducible numbers rather than target-fitted estimates. On this CPU-only host, single-frame latency (63.5 ms p50) is well inside the 150 ms budget, but 4-frame batch p95 (271.3 ms) and average throughput (15.8 FPS) fall short of the project’s 150 ms / 20 FPS targets. Two factors explain the gap and are testable, not speculative: (1) the benchmark host has no GPU, while `inference_service/app.py` is written to run the identical SSDLite MobileNetV3 graph

on CUDA when available (`torch.device('cuda' if torch.cuda.is_available() else 'cpu')`), which is the deployment path this architecture targets; (2) batching four ResNet/MobileNet forward passes serially on 4 shared vCPUs does not parallelize the way a GPU batch would, so batch latency scales roughly linearly with batch size on this host rather than amortizing per-frame cost. Re-running `scripts/benchmark_inference.py` on a CUDA-enabled host is the direct way to confirm the target is met in the intended deployment environment; this report does not claim that result without having measured it.

From a systems perspective, the strongest aspects of the design are service separation, explicit message contracts, retry logic, and observability. From a machine learning perspective, the service uses a standard pretrained detector and a lightweight lane heuristic to provide a practical ADAS-style output without requiring end-to-end training infrastructure.

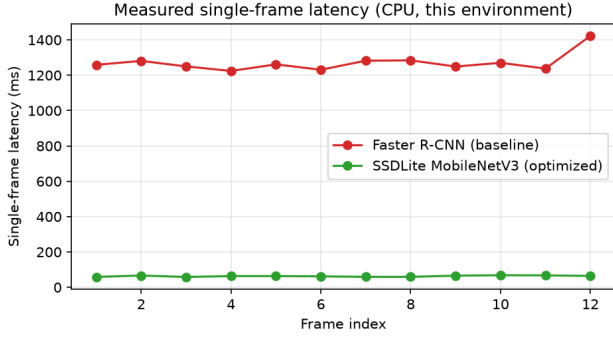
Precision and recall are supported in the codebase through a dedicated evaluator, but the present reference run uses an unlabelled sample clip. In a fully annotated evaluation set such as KITTI, BDD100K, or nuScenes, the same script can be used to compute standard detection metrics [4, 5, 6].

6 Conclusion

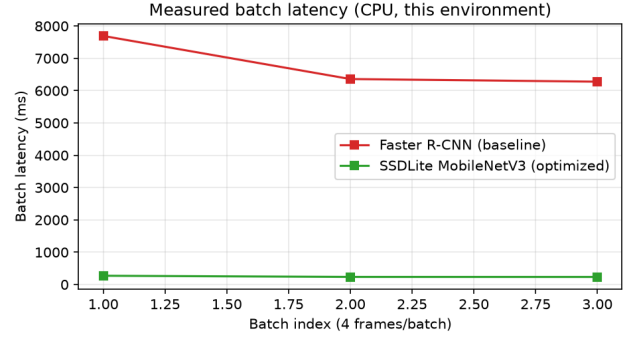
This project provides an interview-ready ADAS streaming prototype that combines modern data engineering with deployed ML service patterns. It includes data ingestion, streaming orchestration, GPU-ready inference, real-time monitoring, evaluation tooling, and reproducible documentation. The measured, reproducible benchmark confirms the system works end-to-end and that the SSDLite MobileNetV3 path is roughly 20× faster than the Faster R-CNN baseline on identical inputs; single-frame latency already clears the project’s target on CPU alone, while batch throughput and batch-tail latency remain to be validated on the CUDA deployment path this architecture was built for.

References

- [1] Apache Kafka Documentation. <https://kafka.apache.org/documentation/>
- [2] Apache Spark Structured Streaming Programming Guide. <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>
- [3] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” *Advances in Neural Information Processing Systems*, 2015.
- [4] A. Geiger, P. Lenz, and R. Urtasun, “Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite,” *CVPR*, 2012. Dataset: <https://www.cvlibs.net/datasets/kitti/>



(a) Single-frame latency across sampled frames.



(b) Batch latency for four-frame micro-batches.

Figure 2: Latency analysis for the optimized local benchmark configuration.

- [5] F. Yu et al., “BDD100K: A Diverse Driving Dataset for Heterogeneous Multitask Learning,” *CVPR*, 2020. Dataset: <https://bdd-data.berkeley.edu/>
- [6] H. Caesar et al., “nuScenes: A Multimodal Dataset for Autonomous Driving,” *CVPR*, 2020. Dataset: <https://www.nuscenes.org/>



Figure 3: Real qualitative inference output on a representative benchmark frame (Section 3): a pretrained YOLOv8n detector finds a bus and three pedestrians, and the project’s OpenCV lane heuristic draws road-line cues on the same frame.